

August 27, 2026

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import glob
import numpy as np
```

```
[2]: sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

from sqlalchemy import create_engine

print('          ...')

#          PostgreSQL
engine = create_engine('postgresql://postgres:Fodruar24533728@localhost:5432/
↳ marketplace_db')

df = pd.read_sql("SELECT * FROM sales WHERE EXTRACT(YEAR FROM order_date) =
↳ 2023", engine)

print(f"          : {len(df):,}")
print(f"          : {df['order_date'].min()} - {df['order_date'].max()}")
print(f"          : {df['customer_id'].nunique():,}")
print(f"          : {df['product_id'].nunique():,}")

df.head()
```

```
...
          : 4,080,152
: 2023-01-01 - 2023-12-31
          : 869,966
          : 50,000
```

```
[2]:   id  customer_id gender  order_date  purchase_time_as_seconds_from_midnight \
0    1      373032      F  2023-01-01                55382
1    2      658080      M  2023-01-01                26969
2    3      387121      F  2023-01-01                85173
3    4      902113      F  2023-01-01                1427
```

4	5	367579	F	2023-01-01	78694
---	---	--------	---	------------	-------

	product_id	quantity	price_per_item	discount_per_item	total_amount	\
0	30569	66	72492.0	26530.0	3033492.0	
1	31098	0	2436.0	1150.0	0.0	
2	1185	62	71950.0	48548.0	1450924.0	
3	34883	3	44981.0	14222.0	92277.0	
4	20898	10	50667.0	29364.0	213030.0	

	price_per_item_rub	discount_per_item_rub	purchase_time	\
0	724.92	265.30	15:23:02	
1	24.36	11.50	07:29:29	
2	719.50	485.48	23:39:33	
3	449.81	142.22	00:23:47	
4	506.67	293.64	21:51:34	

	load_date	data_date	month	quarter	quarter_label
0	2026-08-23 13:19:17.621446	2023-01-01	None	None	None
1	2026-08-23 13:19:17.621446	2023-01-01	None	None	None
2	2026-08-23 13:19:17.621446	2023-01-01	None	None	None
3	2026-08-23 13:19:17.621446	2023-01-01	None	None	None
4	2026-08-23 13:19:17.621446	2023-01-01	None	None	None

```
[3]: df['order_date'] = pd.to_datetime(df['order_date'])

df['quarter'] = df['order_date'].dt.quarter
df['quarter_label'] = df['order_date'].dt.to_period('Q').astype(str)

quarterly_sales = df.groupby('quarter_label')['total_amount'].sum()

plt.figure(figsize=(10,6))
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4']
bars = plt.bar(quarterly_sales.index, quarterly_sales.values, color=colors,
               edgecolor='black', linewidth=1.5)

plt.title('      2023      ', fontsize=16, fontweight='bold')
plt.xlabel('      ', fontsize=12)
plt.ylabel('      ( . )', fontsize=12)
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)
plt.grid(True, alpha=0.3, axis='y')

#
for i, v in enumerate(quarterly_sales.values):
```

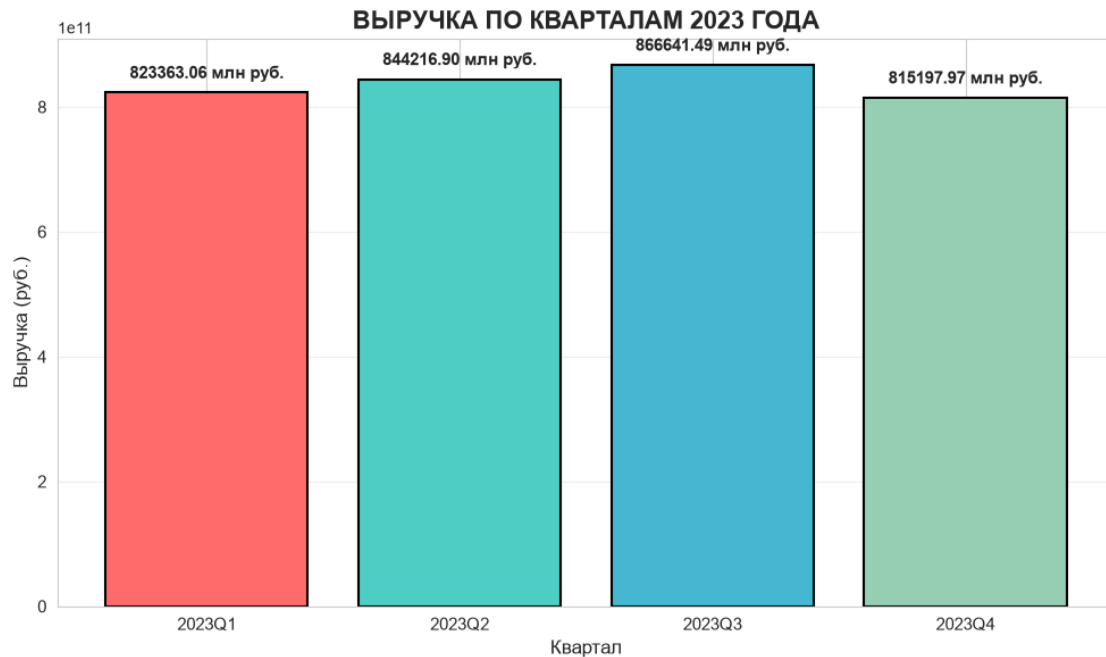
```

plt.text(i,v + v*0.02, f'{v/1000000:.2f}      .',ha='center', va='bottom',
↪ fontsize=11, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n      :")
for q, val in quarterly_sales.items():
    print(f"{q}: {val/1000000:.2f}      .")

print(f"\n      :")
if len(quarterly_sales) == 4:
    growth = ((quarterly_sales.iloc[3] - quarterly_sales.iloc[0]) /
↪ quarterly_sales.iloc[0]) * 100
    print(f"4-      vs 1-      : +{growth:.1f}%")

```



```

:
2023Q1: 823363.06      .
2023Q2: 844216.90      .
2023Q3: 866641.49      .
2023Q4: 815197.97      .

```

```

:
4-      vs 1-      : +-1.0%

```

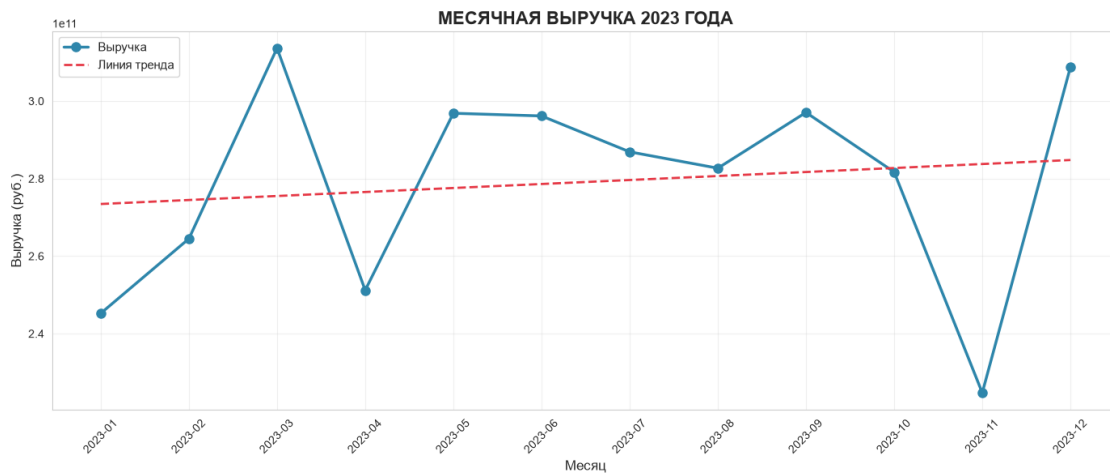
```
[4]: monthly_sales = df.groupby(df['order_date'].dt.to_period('M'))['total_amount'].
      ↪sum()

      month_labels = [str(p) for p in monthly_sales.index]

      plt.figure(figsize=(14,6))
      plt.plot(month_labels, monthly_sales.values, marker='o', linewidth=2.5,
      ↪color='#2E86AB', markersize=8, label='    ')

      #
      z = np.polyfit(range(len(monthly_sales)), monthly_sales.values, 1)
      p = np.poly1d(z)
      plt.plot(month_labels, p(range(len(monthly_sales))), '--', color='#E63946',
      ↪linewidth=2, label='    ')

      plt.title('        2023    ', fontsize=16, fontweight='bold')
      plt.xlabel('    ', fontsize=12)
      plt.ylabel('    ( .) ', fontsize=12)
      plt.xticks(rotation=45, fontsize=10)
      plt.legend(fontsize=11)
      plt.grid(True, alpha=0.3)
      plt.tight_layout()
      plt.show()
```



```
[5]: #        daily_sales (
      daily_sales = df.groupby('order_date')['total_amount'].sum().reset_index()
```

```

#
max_day = daily_sales.loc[daily_sales['total_amount'].idxmax()]
print("                :")
print(f"      : {max_day['order_date']}")
print(f"      : {max_day['total_amount']/1000000:.2f}      .\n")

#
min_day = daily_sales.loc[daily_sales['total_amount'].idxmin()]
print("                :")
print(f"      : {min_day['order_date']}")
print(f"      : {min_day['total_amount']/1000000:.2f}      .\n")

#                (      strptime!)
print("                :")
print(f"{min_day['order_date'].day_name()}")

print("\n                :")
print(f"{max_day['order_date'].day_name()}")

```

```

      :
: 2023-12-08 00:00:00
: 16637.04      .

```

```

      :
: 2023-04-23 00:00:00
: 1567.15      .

```

```

      :
Sunday

```

```

      :
Friday

```

```

[6]: df['order_date'] = pd.to_datetime(df['order_date'])

df['month'] = df['order_date'].dt.month

# 3.
nov_data = df[df['month'] == 11].groupby('order_date')['total_amount'].sum().
    ↪reset_index()

print("                :")
print(f"                : {len(nov_data)}")

#
nov_min = nov_data.loc[nov_data['total_amount'].idxmin()]
print("\n                :")

```

```

print(f"      : {nov_min['order_date']}")
print(f"      : {nov_min['total_amount']/1000000:.2f}      .")
print(f"      : {nov_min['order_date'].day_name()}")

#
nov_max = nov_data.loc[nov_data['total_amount'].idxmax()]
print(f"\n      :")
print(f"      : {nov_max['order_date']}")
print(f"      : {nov_max['total_amount']/1000000:.2f}      .")
print(f"      : {nov_max['order_date'].day_name()}")

#
nov_total = nov_data['total_amount'].sum()
nov_avg = nov_total / len(nov_data)
print(f"\n      : {nov_avg/1000000:.2f}      .")

#
drop_percent = ((nov_avg - nov_min['total_amount']) / nov_avg) * 100
print(f"      {drop_percent:.1f}%")

```

```

:
: 30

:
: 2023-11-03 00:00:00
: 1641.78
: Friday

:
: 2023-11-05 00:00:00
: 16213.24
: Sunday

: 7491.10
78.1%

```

```

[7]: # 1.
nov_dec = df[(df['month'] == 11) | (df['month'] == 12)]

# 2.
daily_nov_dec = nov_dec.groupby('order_date')['total_amount'].sum().
    ↪reset_index()

# 3.
nov_data = daily_nov_dec[pd.to_datetime(daily_nov_dec['order_date']).dt.month_
    ↪== 11]
nov_min = nov_data.loc[nov_data['total_amount'].idxmin()]

```

```

# 4.
dec_data = daily_nov_dec[pd.to_datetime(daily_nov_dec['order_date']).dt.month_
    == 12]
dec_max = dec_data.loc[dec_data['total_amount'].idxmax()]

# 5.
print(f"                : {nov_min['order_date']}")
print(f"    : {nov_min['total_amount']/1000000:.2f}    .")
print(f"    : {nov_min['order_date'].day_name()}\n")

print(f"                : {dec_max['order_date']}")
print(f"    : {dec_max['total_amount']/1000000:.2f}    .")
print(f"    : {dec_max['order_date'].day_name()}\n")

print(f"    : (((dec_max['total_amount'] - nov_min['total_amount']) /
    nov_min['total_amount'] * 100):.0f)%")

# 6.
plt.figure(figsize=(14, 6))

#
plt.plot(daily_nov_dec['order_date'], daily_nov_dec['total_amount'] / 1000000,
    color='#2E86AB', linewidth=2, label='    ')

#
plt.axvline(x=nov_min['order_date'], color='red', linestyle='--', linewidth=2,
    label=f"                : {nov_min['order_date'].strftime('%d.%m')}")

#
plt.axvline(x=dec_max['order_date'], color='green', linestyle='--', linewidth=2,
    label=f"                : {dec_max['order_date'].strftime('%d.%m')}")

#
plt.title('    -    2023:    ', fontsize=14)
plt.xlabel('    ')
plt.ylabel('    (    .)')
plt.xticks(rotation=45)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

```

                : 2023-11-03 00:00:00
: 1641.78    .
    : Friday

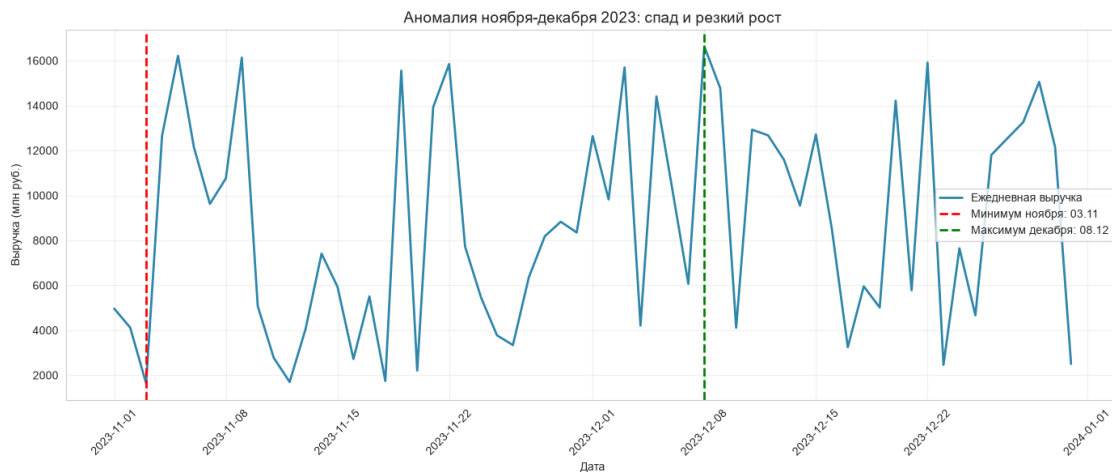
```

```

: 2023-12-08 00:00:00
: 16637.04
: Friday

: 913%

```



```

[8]: # 1.
nov_data = df[df['month'] == 11].copy()
nov_data['order_date'] = pd.to_datetime(nov_data['order_date'])

# 2.
daily_nov = nov_data.groupby('order_date')['total_amount'].sum().reset_index()
daily_nov['day_of_week'] = daily_nov['order_date'].dt.day_name()
daily_nov['day_of_week_ru'] = daily_nov['order_date'].dt.day_name(locale='ru_RU.
↳UTF-8')
daily_nov['day'] = daily_nov['order_date'].dt.day

# 3.
daily_nov = daily_nov.sort_values('order_date')

print(daily_nov[['day', 'day_of_week_ru', 'total_amount']].
↳to_string(index=False))

# 4.
plt.figure(figsize=(16, 6))
bars = plt.bar(daily_nov['day'], daily_nov['total_amount'] / 1000000,
color='#2E86AB', edgecolor='black', linewidth=0.5)

#
for i, row in daily_nov.iterrows():
    if row['day'] % 7 == 0 or row['day'] == 1:

```



```

plt.text(row['day'], -0.5, row['day_of_week_ru'][:3],
         rotation=45, ha='center', fontsize=8, color='gray')

#
avg_nov = daily_nov['total_amount'].mean() / 1000000
plt.axhline(y=avg_nov, color='red', linestyle='--', linewidth=2,
            label=f'                : {avg_nov:.1f}          .')

plt.title('                2023                ', fontsize=14)
plt.xlabel('                ')
plt.ylabel('                (                .)')
plt.xticks(range(1, 32))
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

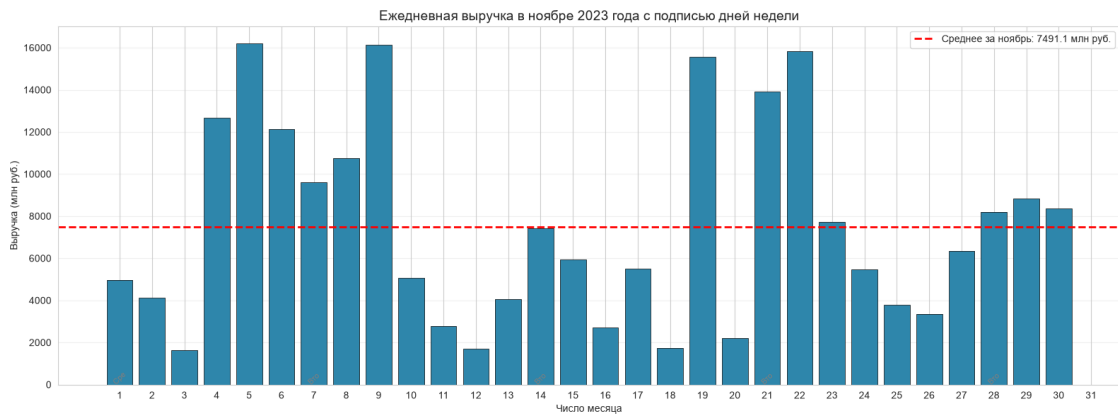
# 5.                :                1000
low_days = daily_nov[daily_nov['total_amount'] < 1000000000] # 1000
print("\n                1000          .")
print(low_days[['day', 'day_of_week_ru', 'total_amount']].
      ↳to_string(index=False))

# 6.                ,                8000
high_days = daily_nov[daily_nov['total_amount'] > 8000000000]
print("\n                8000          .")
print(high_days[['day', 'day_of_week_ru', 'total_amount']].
      ↳to_string(index=False))

```

day	day_of_week_ru	total_amount
1		4.962391e+09
2		4.119222e+09
3		1.641776e+09
4		1.266041e+10
5		1.621324e+10
6		1.213215e+10
7		9.624670e+09
8		1.075947e+10
9		1.613569e+10
10		5.073563e+09
11		2.777032e+09
12		1.698977e+09
13		4.060323e+09
14		7.409137e+09
15		5.937358e+09
16		2.720532e+09
17		5.506380e+09
18		1.739767e+09

19	1.555372e+10
20	2.204256e+09
21	1.392742e+10
22	1.584698e+10
23	7.728854e+09
24	5.463700e+09
25	3.776365e+09
26	3.340050e+09
27	6.359227e+09
28	8.181073e+09
29	8.829911e+09
30	8.349300e+09



```

1000
.
Empty DataFrame
Columns: [day, day_of_week_ru, total_amount]
Index: []

```

```

8000
.
day day_of_week_ru  total_amount
4      1.266041e+10
5      1.621324e+10
6      1.213215e+10
7      9.624670e+09
8      1.075947e+10
9      1.613569e+10
19     1.555372e+10
21     1.392742e+10
22     1.584698e+10
28     8.181073e+09
29     8.829911e+09
30     8.349300e+09

```

2023	: 3	78%	, 5	8.1
.		, 3		,
5	—	.		,

ABC-

```
[9]: product_revenue = df.groupby('product_id')['total_amount'].sum().reset_index()
product_revenue.columns = ['product_id', 'revenue']

product_revenue = product_revenue.sort_values('revenue', ascending=False)

#
product_revenue['cumulative_revenue'] = product_revenue['revenue'].cumsum()
total_revenue = product_revenue['revenue'].sum()
product_revenue['cumulative_share'] = product_revenue['cumulative_revenue']/
    ↪total_revenue * 100

def assing_abc(share):
    if share <= 70:
        return 'A'
    elif share <= 90:
        return 'B'
    else:
        return 'C'

product_revenue['abc_category'] = product_revenue['cumulative_share'].
    ↪apply(assing_abc)

abc_summary = product_revenue.groupby('abc_category').agg({
    'product_id': 'count',
    'revenue': 'sum'
}).rename(columns={
    'product_id': 'number_of_products',
    'revenue': 'total_revenue'
})

abc_summary['revenue_share'] = (abc_summary['total_revenue'] / total_revenue *
    ↪100)
abc_summary['total_revenue'] = abc_summary['total_revenue']

print(abc_summary)
```

	number_of_products	total_revenue	revenue_share
abc_category			
A	21856	2.344574e+12	69.999406
B	11867	6.698799e+11	19.999882
C	16277	3.349658e+11	10.000711

```
[10]: for category in ['A', 'B', 'C']:
        sample = product_revenue[product_revenue['abc_category'] == category].
        ↪head(5)
        print(f"\n          {category}:")
        for _, row in sample.iterrows():
            print(f"          {row['product_id']}: {row['revenue']} .")
```

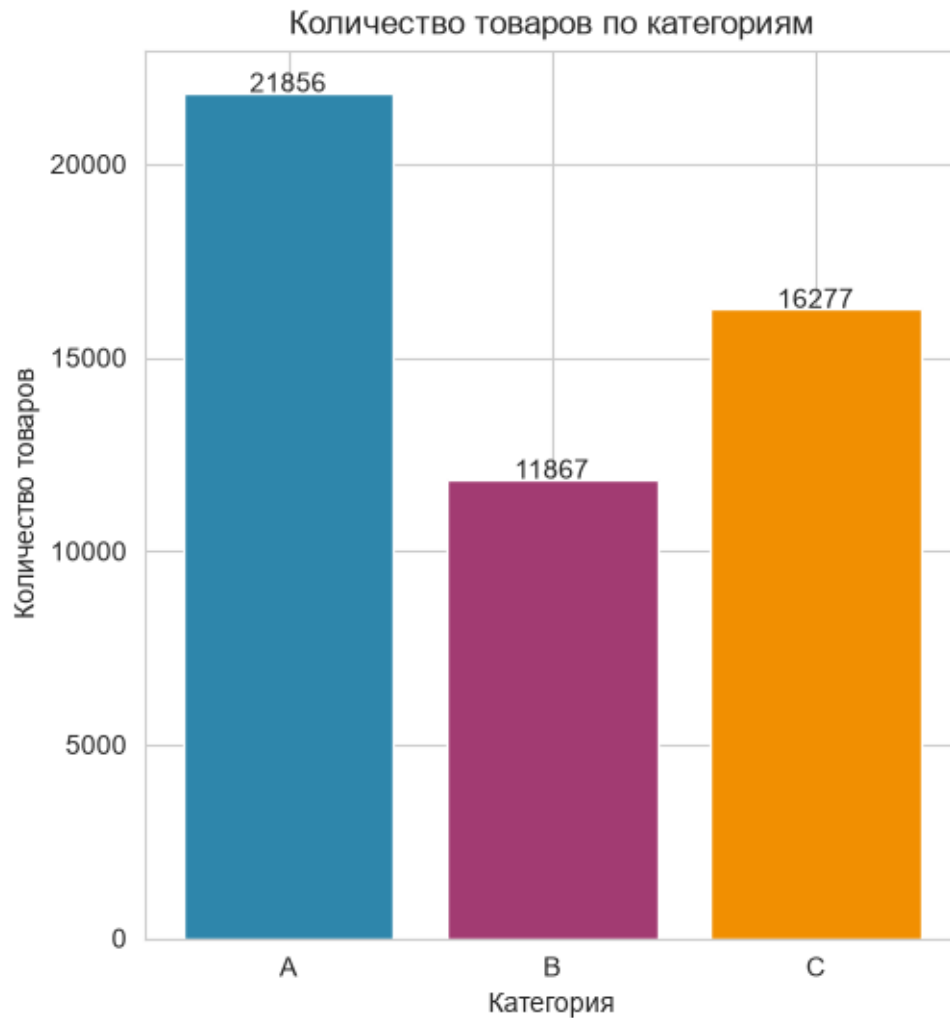
```
A:
37181: 253013852.0 .
30239: 245154170.0 .
22406: 244015782.0 .
10292: 242230062.0 .
45018: 236644456.0 .
```

```
B:
44558: 71893182.0 .
44219: 71887236.0 .
26069: 71887002.0 .
29960: 71880252.0 .
40855: 71879814.0 .
```

```
C:
34757: 40937366.0 .
7275: 40928924.0 .
32730: 40927954.0 .
37805: 40925572.0 .
44253: 40925350.0 .
```

```
[11]: plt.figure(figsize=(12, 6))

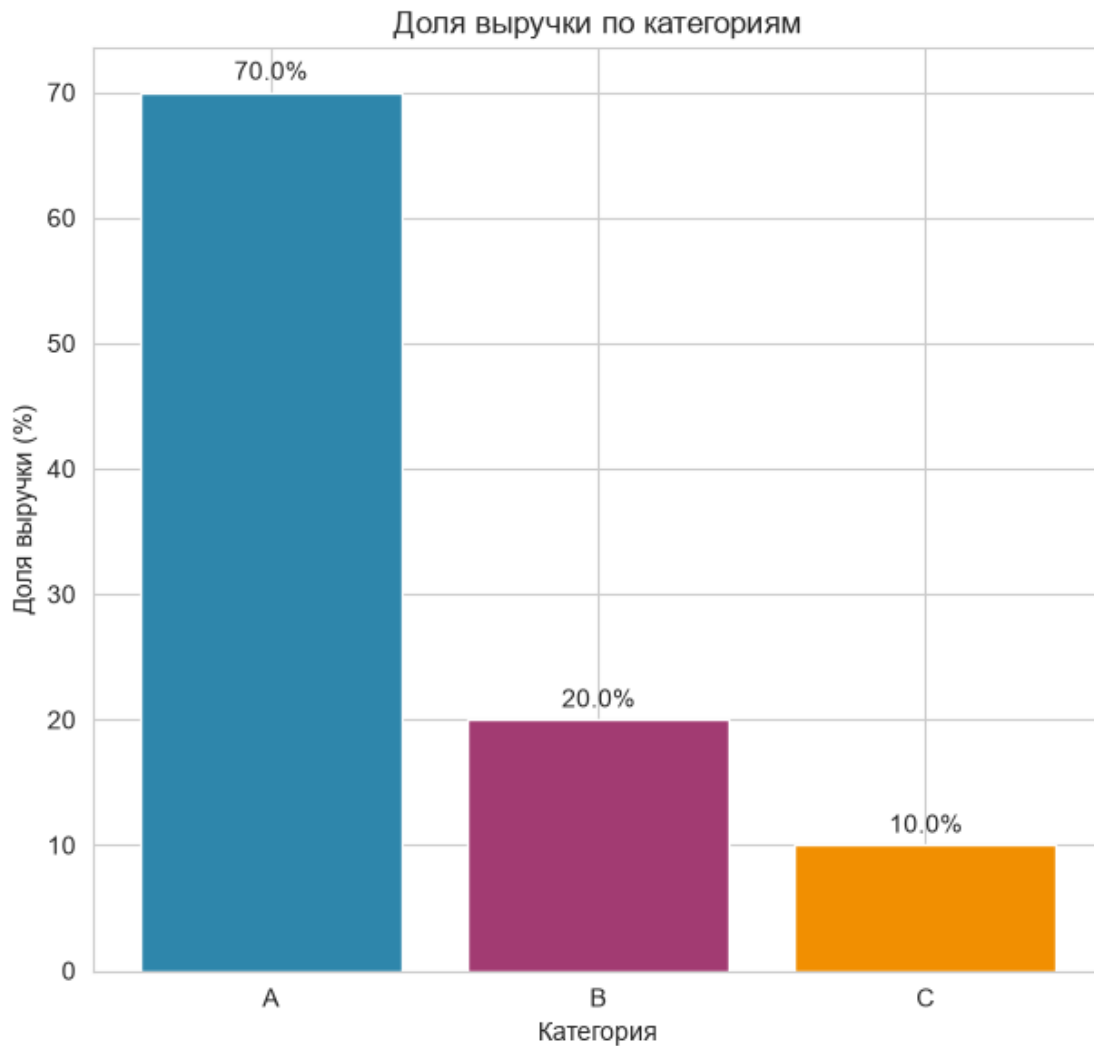
#      1:
plt.subplot(1, 2, 1)
category_counts = product_revenue['abc_category'].value_counts().sort_index()
bars = plt.bar(category_counts.index, category_counts.values, color=['#2E86AB', '
        ↪'#A23B72', '#F18F01'])
plt.title('          ')
plt.xlabel('          ')
plt.ylabel('          ')
for bar, count in zip(bars, category_counts.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5, str(count),
        ↪ha='center')
```



```
[12]: # 2:
plt.subplot(1, 2, 2)
revenue_by_category = product_revenue.groupby('abc_category')['revenue'].sum().
    ↪sort_index()
revenue_share = revenue_by_category / total_revenue * 100
bars = plt.bar(revenue_share.index, revenue_share.values, color=['#2E86AB', '
    ↪'#A23B72', '#F18F01'])
plt.title('')
plt.xlabel('')
plt.ylabel(' (%)')
for bar, share in zip(bars, revenue_share.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1, f'{share:.
    ↪1f}% ', ha='center')

plt.tight_layout()
```

```
plt.show()
```



```
[13]: # 7.
for category in ['A', 'B', 'C']:
    cat_data = product_revenue[product_revenue['abc_category'] == category]
    print(f"      {category}:")
    print(f"      : {len(cat_data)}")
    print(f"      : {cat_data['revenue'].mean()} .")
    print(f"      : {cat_data['revenue'].min()} .")
    print(f"      : {cat_data['revenue'].max()} .")
```

```
A:
: 21856
: 107273686.93045388 .
: 71899484.0 .
```

: 253013852.0 .
 B:
 : 11867
 : 56448971.026375666 .
 : 40938806.0 .
 : 71893182.0 .
 C:
 : 16277
 : 20579085.023038644 .
 : 1308.0 .
 : 40937366.0 .
 ABC- 70/20/10 :
 A 21 856 53,6 . 70% . ,
 B 11 867 28,2 . 20% .
 A
 C 16 277 10,3 . 10% .
 654 . , ,
 , , , A B,
 C : , , .
 10 000 .
 LTV

```
[14]: customer_ltv = df.groupby('customer_id').agg({'total_amount': 'sum',
↪ 'order_date': ['min', 'max'], 'product_id': 'count'}).reset_index()
customer_ltv
```

```
[14]:
```

	customer_id	total_amount	order_date		product_id
		sum	min	max	count
0	0	2605716.0	2023-12-14	2023-12-14	2
1	2	42020.0	2023-11-05	2023-11-05	2
2	3	2199006.0	2023-02-14	2023-08-27	10
3	4	33648.0	2023-03-09	2023-03-09	2
4	5	2022708.0	2023-01-19	2023-03-04	4
...	...	...	...	...	...
869961	999994	10044510.0	2023-05-22	2023-09-12	4
869962	999995	3268654.0	2023-06-11	2023-06-11	2
869963	999996	535898.0	2023-07-26	2023-10-30	4
869964	999997	2347858.0	2023-09-02	2023-09-19	4
869965	999998	7431952.0	2023-04-03	2023-12-22	8

[869966 rows x 5 columns]

```
[15]: #
customer_ltv = df.groupby('customer_id').agg({
    'total_amount': 'sum',
    'order_date': ['min', 'max'],
    'product_id': 'count' # =
}).reset_index()

#
customer_ltv.columns = ['customer_id', 'total_spent', 'first_order',
    ↪ 'last_order', 'orders']

#
customer_ltv['lifetime_days'] = (customer_ltv['last_order'] -
    ↪ customer_ltv['first_order']).dt.days

#LTV =
customer_ltv['ltv'] = customer_ltv['total_spent']

#
customer_ltv['avg_order_value'] = customer_ltv['total_spent'] /
    ↪ customer_ltv['orders']
customer_ltv['orders_per_month'] = customer_ltv['orders'] /
    ↪ (customer_ltv['lifetime_days'] / 30)

#
customer_ltv['orders_per_month'] = customer_ltv['orders_per_month'].
    ↪ replace([float('inf'), -float('inf')], 0)

#
print(f"      : {len(customer_ltv):}")
print(f"      : {customer_ltv['total_spent'].sum()/1000000:.2f}  "
    ↪ ".")
print(f"      LTV      : {customer_ltv['ltv'].mean():.0f}  .")
print(f"      LTV: {customer_ltv['ltv'].median():.0f}  .")
print(f"      LTV: {customer_ltv['ltv'].min()}  .")
print(f"      LTV: {customer_ltv['ltv'].max():.0f}  .")
print(f"\n      : {customer_ltv['orders'].mean():.2f}")
print(f"      : {customer_ltv['avg_order_value'].mean():.0f}  .")
```

```
      : 869966
      : 3349419.41
LTV      : 3850058
LTV: 2633491
LTV: 0.0
LTV: 43477290

      : 4.69
```


: 820226 .

```
[16]: # 1.      LTV
customer_ltv = df.groupby('customer_id').agg({
    'total_amount': 'sum',
    'order_date': ['min', 'max'],
    'product_id': 'count'
}).reset_index()

customer_ltv.columns = ['customer_id', 'total_spent', 'first_order',
    ↪ 'last_order', 'orders']
customer_ltv['ltv'] = customer_ltv['total_spent']

# 2.      (      95%      )
q95 = customer_ltv['ltv'].quantile(0.95)
filtered = customer_ltv[customer_ltv['ltv'] <= q95]

# 3.
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

#      1:      (      )
axes[0].boxplot(filtered['ltv'] / 1000000, vert=True, patch_artist=True,
    boxprops=dict(facecolor='#2E86AB', color='black'),
    whiskerprops=dict(color='black'),
    capprops=dict(color='black'),
    medianprops=dict(color='red', linewidth=2))
axes[0].set_title('      LTV ( 95-      )', fontsize=14)
axes[0].set_ylabel('LTV (      .)')
axes[0].set_xticklabels(['LTV'])
axes[0].grid(True, alpha=0.3, axis='y')

#
mean_val = filtered['ltv'].mean() / 1000000
median_val = filtered['ltv'].median() / 1000000
axes[0].scatter(1, mean_val, color='green', s=100, zorder=5, label=f'      :␣
    ↪ {mean_val:.2f}      ')
axes[0].scatter(1, median_val, color='orange', s=100, zorder=5, label=f'      :␣
    ↪ {median_val:.2f}      ')
axes[0].legend()

#      2:      LTV
axes[1].hist(filtered['ltv'] / 1000000, bins=50, color='#A23B72',
    ↪ edgecolor='black', alpha=0.7)
axes[1].axvline(mean_val, color='green', linestyle='--', linewidth=2,
    ↪ label=f'      : {mean_val:.2f}      ')
axes[1].axvline(median_val, color='orange', linestyle='--', linewidth=2,
    ↪ label=f'      : {median_val:.2f}      ')
```

```

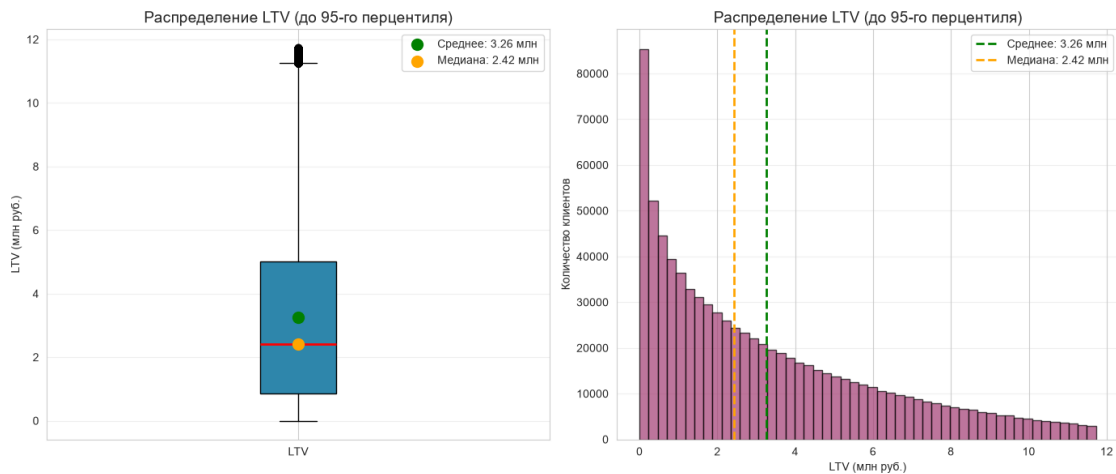
axes[1].set_title('LTV ( 95- )', fontsize=14)
axes[1].set_xlabel('LTV ( .)')
axes[1].set_ylabel(' ')
axes[1].legend()
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# 4.
print(f" LTV: {filtered['ltv'].mean():.0f} .")
print(f" LTV: {filtered['ltv'].median():.0f} .")
print(f" : {df.groupby('customer_id')['total_amount'].sum().mean():.0f} .
↪")
print(f" LTV: {(filtered['ltv'].mean() - filtered['ltv'].
↪median()):.0f} .")

```

C:\Users\kozya\AppData\Local\Temp\ipykernel_6092\3639980711.py:19:
MatplotlibDeprecationWarning: vert: bool was deprecated in Matplotlib 3.11 and
will be removed in 3.13. Use orientation: {'vertical', 'horizontal'} instead.
axes[0].boxplot(filtered['ltv'] / 1000000, vert=True, patch_artist=True,



```

LTV: 3261818 .
LTV: 2421410 .
: 3850058 .
LTV: 840408 .
, LTV LTV .
,

```

RFM-

R (Recency) —

F (Frequency) —

M (Monetary) —

:

Champions - , (,)

Loyal - , (,)

Potential - , ()

At Risk - , (,)

Lost - ()

```
[17]: # 1.
current_date = df['order_date'].max()

rfm_table = df.groupby('customer_id').agg({
    'order_date': lambda x: (current_date - x.max()).days,
    'product_id': 'count',
    'total_amount': 'sum'
})

rfm_table.columns = ['recency', 'frequency', 'monetary']

# ID ( )
rfm_table = rfm_table.reset_index()

print(" . :", list(rfm_table.columns))

# 2.
rfm_table['r_score'] = pd.qcut(rfm_table['recency'].rank(method='first'), q=4,
    ↪ labels=[4, 3, 2, 1]).astype(int)
rfm_table['f_score'] = pd.qcut(rfm_table['frequency'].rank(method='first'),
    ↪ q=4, labels=[1, 2, 3, 4]).astype(int)
rfm_table['m_score'] = pd.qcut(rfm_table['monetary'].rank(method='first'), q=4,
    ↪ labels=[1, 2, 3, 4]).astype(int)

# 3.
def get_segment(r, f, m):
    if r >= 4 and f >= 4 and m >= 3:
        return 'Champions'
    elif r >= 3 and f >= 3 and m >= 3:
        return 'Loyal'
    elif r >= 4 and m >= 3 and f <= 2:
        return 'Potential Loyalists'
    elif r <= 2 and m >= 3:
        return 'At Risk'
    elif r <= 2 and f <= 2:
```

```

        return 'Lost'
    else:
        return 'Other'

rfm_table['segment'] = rfm_table.apply(lambda x: get_segment(x['r_score'],
    ↪x['f_score'], x['m_score']), axis=1)

print(f"                : {len(rfm_table):,}")

.      : ['customer_id', 'recency', 'frequency', 'monetary']
.      : 869,966

```

```

[18]: #
segment_summary = rfm_table.groupby('segment').agg({
    'customer_id': 'count',
    'monetary': 'sum',
    'recency': 'mean',
    'frequency': 'mean'
}).rename(columns={'customer_id': 'clients'})

segment_summary['revenue_share'] = (segment_summary['monetary'] /
    ↪rfm_table['monetary'].sum() * 100).round(1)

print(segment_summary.sort_values('revenue_share', ascending=False))

#
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

segments = segment_summary.index
counts = segment_summary['clients'].values
axes[0].bar(segments, counts, color='#2E86AB')
axes[0].set_title('')
axes[0].tick_params(axis='x', rotation=45)

revenue_share = segment_summary['revenue_share'].values
axes[1].bar(segments, revenue_share, color='#A23B72')
axes[1].set_title(' (%)')
axes[1].tick_params(axis='x', rotation=45)

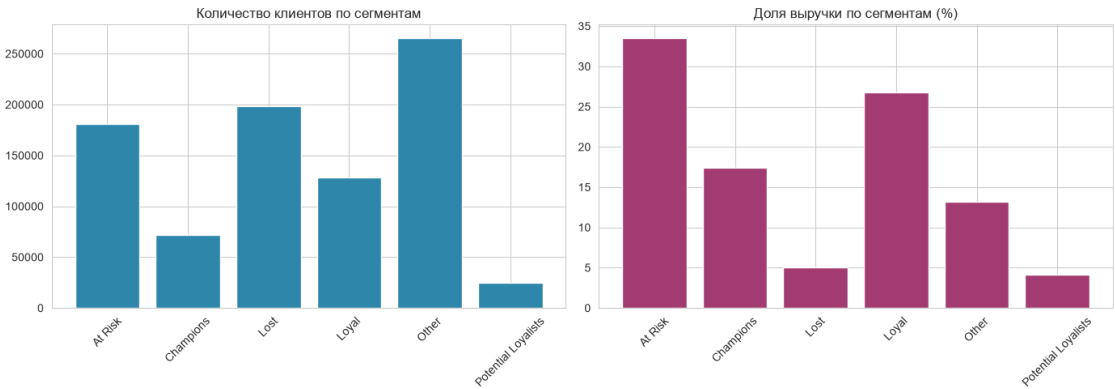
plt.tight_layout()
plt.show()

```

	clients	monetary	recency	frequency \
segment				
At Risk	181045	1.121473e+12	186.693225	4.863620
Loyal	128651	8.991997e+11	57.748785	6.699629
Champions	71904	5.823834e+11	19.072931	8.719821
Other	265292	4.422475e+11	78.390359	4.273073

Lost	198451	1.678217e+11	221.811409	2.479030
Potential Loyalists	24623	1.362946e+11	20.480567	3.457580

revenue_share	
segment	
At Risk	33.5
Loyal	26.8
Champions	17.4
Other	13.2
Lost	5.0
Potential Loyalists	4.1



RFM- 6 .

Champions Loyal, 23% 44.2% .

(4.36 3.35) (19 58) .

At Risk, 20.8% 33.5% . LTV,

187 . Lost.

(15–20%) .

Lost (22.8%) 5% .

Potential Loyalists (2.8%) — .

Champions.